GURU GHASIDAS VISHWAVIDYALAYA

(A Central University established by the Central Universities Act, 2009 No. 25 of 2009)

Bilaspur, Chhattisgarh – 495009

[INFORMATION TECHNOLOGY]



OOPS PROJECT REPORT

(Object-Oriented Programming)

on

TRADING JOURNAL:

High-Performance C++17 Web Application

Submitted by

Student Name	Roll Number
Utkarsh Kumar	25008288 (41)
Pinkesh Kumar Mishra	25008253 (07)
Rabeekant	25008259 (13)

Under the guidance of

[Suhel Ahmed], [Assistant Professor]

Programme: BTECH,IT **Academic Session: 2025-29**

Date of Submission: 28-10-2026

GURU GHASIDAS VISHWAVIDYALAYA

Bilaspur, Chhattisgarh

CERTIFICATE

This is to certify that the OOPS Mini project report entitled **"Trading Journal: High-Performance C++17 Web Application"** is a bonafide record of the Object-Oriented Programming Mini project work carried out by

- **Utkarsh Kumar (Roll No. 25008288 (41))**
- **Pinkesh Kumar Mishra (Roll No. 25008253 (07))**
- **Rabeekant (Roll No. 25008259 (13))**

students of B-tech [IT] , Guru Ghasidas Vishwavidyalaya, Bilaspur, during the academic session 2025-29, under my guidance and supervision. To the best of my knowledge, the work has been carried out by the students and has been submitted for the OOPS Mini Project of the said programme.

INDEX

Certificate **2**
Index **3**
1. Introduction **4**
1.1 Overview of the Project 4
1.2 Background of Trading Journals 4
1.3 Problem Statement 4
1.4 Objectives 4
1.5 Technologies at a Glance 5
1.6 Organisation of the Report 5
2. Motivation **6**
2.1 Why Record and Analyse Trades 6
2.2 Problems of Manual Tracking 6
2.3 Academic Motivation 6
2.4 Existing Approach versus Proposed System 6
2.5 Intended Users and Significance 7
3. Design **8**
3.1 Requirement Analysis 8
3.2 Technology Stack 9
3.3 Layered Architecture 9
3.4 Request Lifecycle and Workflow 10
3.5 Database Design 11
3.6 Object-Oriented Class Design 13
3.7 Mathematical Design of the Analytics 15
3.8 Security Design 15
4. Implementation **16**
4.1 Project Directory Structure 16
4.2 Module Organisation and File Responsibilities 16
4.3 Domain Layer Implementation 17
4.4 Journal and Template-Based Aggregation 18
4.5 Data Layer and RAII Wrappers 18
4.6 Web and Security Layer 18
4.7 Object-Oriented Concepts Applied 19
4.8 Build and Execution 19
4.9 Team Contributions 20
5. Result **21**
5.1 Features Delivered 21
5.2 Worked Example of the Analytics 22
5.3 Test Plan and Expected Results 23
5.4 Discussion 24
6. Conclusion **25**
7. Scope and Limitations **26**
7.1 Scope of the Project 26
7.2 Limitations 26
7.3 Future Enhancements 26
References **27**
Appendix **28**
Appendix A: SQL Schema (sql/schema.sql) 28
Appendix B: Important Formulas 28
Appendix C: Selected C++ Excerpts 28
Appendix D: Points to Confirm Before Final Submission 29
GitHub Repository **30**

1. INTRODUCTION

1.1 Overview of the Project

The **Trading Journal** is a multi-user web application whose backend is written in modern C++17. It allows a trader to record every trade (ticker, direction, entry and exit price, quantity, fees, date and notes), to review the history in a filterable ledger, and to open a dashboard that shows Total P&L, Win Rate, Profit Factor, Maximum Drawdown, Average Win and Average Loss together with an equity curve. Trades can be exported as a standard CSV file, and every user sees only his or her own data.

The project was developed as the OOPS (Object-Oriented Programming) Mini project of the course. Its main aim is not to predict markets or to give investment advice; it is to demonstrate, inside one realistic program, how the principles of object-oriented design (encapsulation, abstraction, inheritance and runtime polymorphism) and modern C++ resource management (RAII, smart pointers and move semantics) are used to build a clean, layered and secure application.

1.2 Background of Trading Journals

A trading journal is a structured record of the trades a person has made in financial markets. Each entry normally captures what was traded, in which direction (buying first is called **Long**, selling first is called **Short**), at what entry and exit prices, in what quantity, at what cost, and on which date. Free-text notes about the reasoning behind a trade are often added.

Memory is a poor record of trading performance. Profitable trades tend to be remembered more vividly than small repeated losses, and a few large outcomes can distort a trader's impression of how he or she is doing. A written history summarised with consistent statistics replaces impressions with figures: total profit and loss, the proportion of winning trades, the ratio of gross profits to gross losses, and the deepest peak-to-trough fall in cumulative results. Reviewing such figures helps a user judge how past trades performed. This report makes no claim that using the software improves financial results or guarantees profit.

1.3 Problem Statement

To design and implement a multi-user, browser-accessible trade journal in modern C++17 that stores each user's trades persistently, calculates trade-level and portfolio-level performance statistics from the recorded history, presents them visually, and does so using sound object-oriented design, deterministic resource management and basic security practices.

1.4 Objectives

The project has five objectives:

1. **Object-oriented design:** apply encapsulation, abstraction, runtime polymorphism and const-correctness to financial analytics.
2. **Resource management:** use RAII and move semantics so that resources are released automatically, without a garbage collector.
3. **Decoupled architecture:** separate presentation, domain and data concerns following the Dependency Inversion Principle.
4. **Security and data isolation:** support safe multi-user operation through Argon2id hashing, parameterized SQL and session-guarded routes.
5. **Memory safety and verification:** use clean resource deallocation, and the AddressSanitizer and UndefinedBehaviorSanitizer tools, as verification aids.

1.5 Technologies at a Glance

The backend uses the **Crow** C++ microframework for HTTP routing, middleware and Mustache view templates, **SQLite3** for persistence through its C API, and **libsodium** for Argon2id password hashing. The frontend is plain **HTML5** with **Bootstrap 5** for layout and **Chart.js** for the equity curve. The project is built with **CMake** (version 3.16 or later) and can be checked with the AddressSanitizer and UndefinedBehaviorSanitizer compiler options. Chapter 3 explains why each technology was chosen.

1.6 Organisation of the Report

- **Chapter 2 (Motivation)** explains why the project was undertaken and compares manual tracking with the proposed system.
- **Chapter 3 (Design)** presents the requirements, technology stack, layered architecture, database design, class design, mathematical design and security design.
- **Chapter 4 (Implementation)** describes the file structure, the key classes and code excerpts, the OOP concepts applied, the build procedure and the contribution of each team member.
- **Chapter 5 (Result)** reports the features delivered, a worked numerical example of the analytics, the test plan and the screenshot plan.
- **Chapter 6 (Conclusion)** and **Chapter 7 (Scope and Limitations)** close the report with the outcome, the limits of the work and possible future extensions.
- The **References**, the **Appendix** and the **GitHub repository link** follow at the end.

2. MOTIVATION

2.1 Why Record and Analyse Trades

Anyone who trades regularly eventually needs answers to simple questions: Am I profitable overall? How often do my trades win? Do my winners outweigh my losers? What was the worst losing stretch? These questions cannot be answered reliably from memory, and answering them from loose notes means repeating the same arithmetic after every trade. A journal that calculates the figures automatically from a validated history removes that effort and removes the chance of calculation mistakes.

2.2 Problems of Manual Tracking

Notebooks, text files and general-purpose spreadsheets are the usual tools. They are flexible, but flexibility is also their weakness:

- Profit and loss must be recomputed by hand, and the formula differs between Long and Short trades, which invites arithmetic errors.
- Entries may have inconsistent formats, missing fields or impossible values, such as a negative quantity.
- Statistics such as drawdown need an ordered running calculation that is tedious to repeat each time a trade is added.
- Formulas in a spreadsheet can be overwritten accidentally, and the same calculation may be implemented differently in different files.
- Records kept on one device are hard to protect, hard to share selectively and hard to export cleanly.
- When several people need a journal, a shared file provides no separation between their data.

2.3 Academic Motivation

The Object-Oriented Programming course introduces classes, inheritance, polymorphism, templates, exceptions and resource management. These ideas are understood best when they solve a real design problem instead of isolated exercises. A financial analytics engine is a natural place to apply an abstract metric interface (the Strategy pattern), a repository abstraction that hides the database (the Repository pattern), and resource-owning classes that must be move-only. A small web application also forces the three team members to agree on interfaces, which makes the project a lesson in collaborative software engineering as well.

2.4 Existing Approach versus Proposed System

Table 2.1: Manual records versus the proposed Trading Journal

Aspect	Manual or unstructured records	Trading Journal (as designed)
Profit and loss	Computed by hand or with user-written formulas	Computed by the `Trade` class for Long and Short trades
Input checking	Depends on the user	Server-side validators plus database CHECK constraints
Statistics	Recomputed manually	Metric classes behind the `IMetric` interface
Visualisation	Manual chart creation	Equity curve rendered with Chart.js
Multiple users	Separate files or one shared file	User accounts with user-scoped queries
Storage	Loose files	SQLite relational database with foreign keys
Export	Copying cells	RFC 4180 CSV export

The comparison is between design properties. This report does not compare the project with any specific commercial product and does not claim a performance advantage, because no benchmark was measured.

2.5 Intended Users and Significance

The intended users are individual traders in cash equities and similar linear instruments who want to keep and review their own records, and, in the academic context, evaluators and students who want to study a worked example of object-oriented C++17 design. For the team the significance lies in integration: three separately developed modules (domain and analytics, data persistence, web and security) had to combine into one running program, so the project demonstrates both individual OOP concepts and teamwork.

3. DESIGN

3.1 Requirement Analysis

The functional requirements describe what the system must do. Each requirement is traceable to a module described in Chapter 4.

Table 3.1: Functional requirements

ID	Requirement
FR1	A visitor shall be able to register an account with a username and password.
FR2	A registered user shall be able to log in and log out.
FR3	A logged-in user shall be able to add a trade with ticker, direction (Long or Short), entry price, exit price, quantity, fees, date and notes.
FR4	A user shall be able to edit an existing trade of their own.
FR5	A user shall be able to view their trade history as a ledger.
FR6	A user shall be able to filter trades by ticker symbol and by outcome (wins or losses).
FR7	The system shall calculate profit and loss per trade using the Long or Short formula.
FR8	The system shall validate trade inputs before storage.
FR9	The system shall display Total P&L, Win Rate, Profit Factor, Maximum Drawdown, Average Win and Average Loss.
FR10	The system shall display a cumulative equity curve.
FR11	A user shall be able to export their trades as CSV.
FR12	A user shall only be able to access their own trades.
FR13	All data shall persist in a SQLite database across application restarts.

The non-functional requirements are design goals. They are listed with the way each is addressed in the design.

Table 3.2: Non-functional requirements

ID	Requirement	How the design addresses it
NFR1	Modularity	Three isolated layers; the domain classes include no Crow or SQLite headers.
NFR2	Deterministic resource release	RAII wrappers `Database` and `Statement` release handles in destructors.
NFR3	Security	Argon2id hashing, parameterized SQL, `AuthGuard` session middleware, user-scoped queries.
NFR4	Portability	CMake build for Linux and macOS with documented prerequisites.
NFR5	Responsive interface	Bootstrap 5 grid, tables and forms.
NFR6	Testability	`InMemoryTradeRepository` allows logic to be exercised without a database.
NFR7	Data integrity	NOT NULL, UNIQUE, CHECK and FOREIGN KEY constraints in the schema.

3.2 Technology Stack

Table 3.3: Technologies and the reason for their selection

Technology	Role in the project	Reason for selection
C++17	Entire backend	Modern idioms (`std::optional`, smart pointers, lambdas), type safety and deterministic destruction.
Crow	Routing, middleware, cookies, templates	Lightweight; routing, HTTP/1.1 parsing, session middleware and Mustache in one header-only framework.
SQLite3	Persistence of users and trades	Serverless, zero-configuration and transactional; accessible through a C API.
libsodium	Password hashing (Argon2id)	Memory-hard hashing that is resistant to GPU and ASIC attacks.
SQL	Schema, seed data, queries	Standard language for relational data.
HTML5, Mustache	Page structure and templates	Standard markup; Mustache escapes values by default.
Bootstrap 5	Styling	Responsive layouts, tables, forms and alerts.
Chart.js	Equity curve	Client-side canvas charts for time series.
CMake 3.16+	Build system	Standard cross-platform configuration.
ASan / UBSan	Run-time error detection	Optional sanitizer build used as a verification aid.

Software requirements. Linux (Ubuntu, Debian or WSL) or macOS; a C++17 compiler (GCC or Clang); CMake 3.16 or later; the packages `build-essential`, `cmake`, `libsqlite3-dev`, `libsodium-dev`, `libasio-dev` and `sqlite3` on Ubuntu or Debian, or `cmake`, `sqlite`, `libsodium` and `asio` through Homebrew on macOS; and a current web browser with JavaScript and HTML5 canvas support. Crow itself is bundled as `third_party/crow/crow_all.h`.

3.3 Layered Architecture

The central design decision is to keep presentation, domain and data concerns in separate layers. The browser talks only to the web layer, the web layer talks to the domain and data layers through interfaces, and only the data layer touches the SQLite file. Figure 3.1 shows the five levels that together realise the three-layer objective.

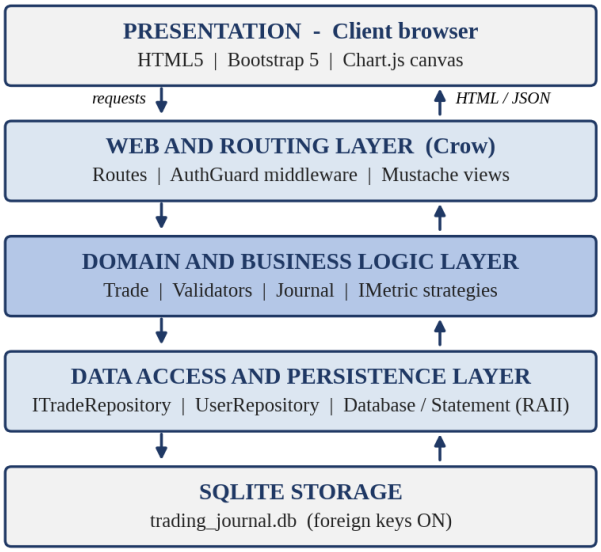


Figure 3.1: Layered system architecture

1. **Presentation layer.** The browser renders HTML5, Bootstrap 5 and a Chart.js canvas, sends HTTP requests and displays the responses.
2. **Web and routing layer.** The Crow application: route dispatchers, the `AuthGuard` middleware that checks the session cookie, and Mustache view rendering.
3. **Domain and business logic layer.** The `Trade` value object, validators with `ValidationError`, the move-only `Journal`, the `IMetric` strategies, the equity-curve calculation and the templated `sumProfit` algorithm. It has no dependency on Crow or SQLite.
4. **Data access and persistence layer.** The `ITradeRepository` interface with `SqliteTradeRepository` and `InMemoryTradeRepository`, the `UserRepository`, and the RAII wrappers `Database` and `Statement`.
5. **SQLite database storage.** The file `trading_journal.db` with foreign-key enforcement enabled.

This arrangement gives each layer one reason to change, lets high-level code depend on abstractions (`IMetric`, `ITradeRepository`) instead of concrete classes, and means that the analytics code can be examined without any web or database code.

3.4 Request Lifecycle and Workflow

The browser sends an HTTP request to the Crow server, which matches the URL to a route. For protected routes the `AuthGuard` middleware runs first and redirects an unauthenticated visitor to the login page; the route handler then calls the lower layers through the injected `Deps`

structure. For a trade submission, the handler passes the form values to the validators, which throw `ValidationError` for invalid input. Valid data becomes a `Trade` object that is handed to the `ITradeRepository`. The SQLite implementation opens a `Statement` through the `Database` wrapper, binds the values (including the user's id) as parameters and executes the SQL. For the dashboard, the handler loads the user's trades, builds a `Journal`, calls `computeAll()` to obtain each metric, and renders the values into a Mustache template; the equity curve is delivered separately as JSON, which `charts.js` fetches and draws. Figure 3.2 summarises the user-level workflow.

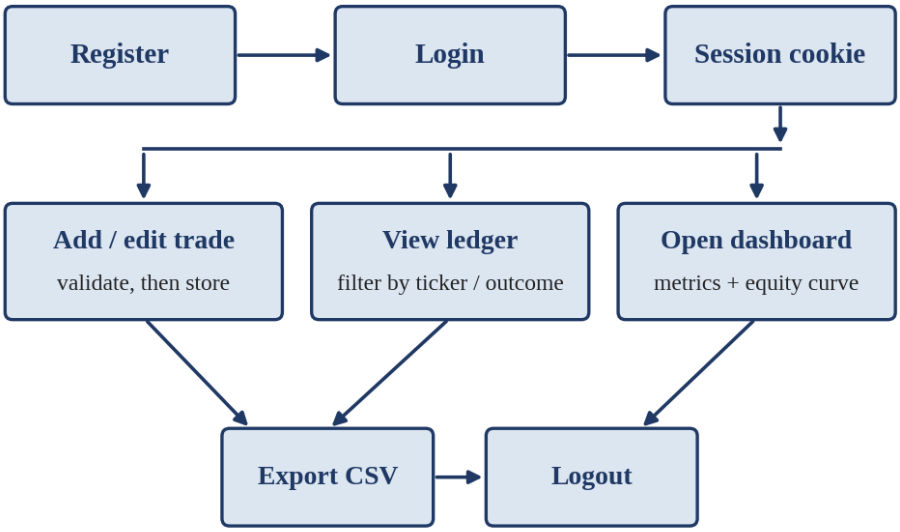


Figure 3.2: High-level application workflow

3.5 Database Design

SQLite is a serverless, zero-configuration relational database that keeps all data in one file and supports transactions. For a locally run academic application it avoids installing a separate database server while still providing tables, constraints, indexes and SQL. The design uses two tables in a one-to-many relationship: each user owns many trades (Figure 3.3). The complete schema is reproduced in Appendix A.

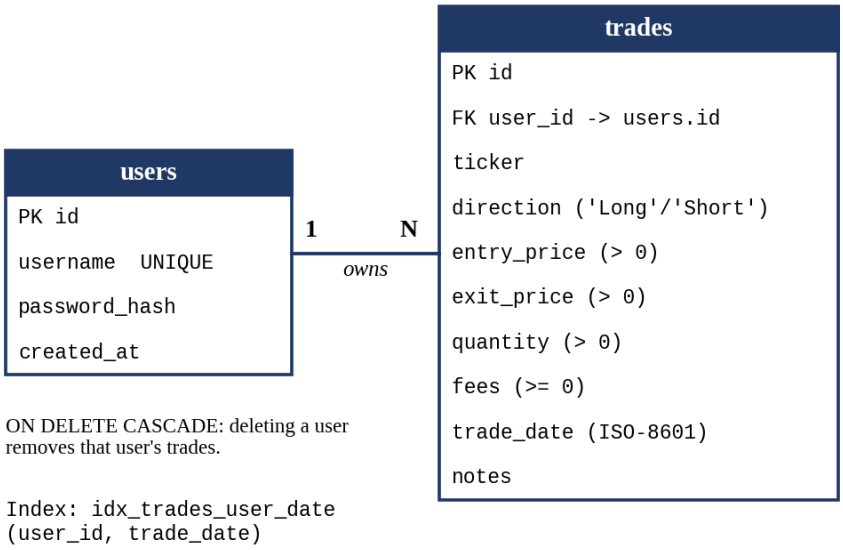


Figure 3.3: Entity-relationship diagram

The `users` table stores `id`, a unique `username`, the `password_hash` (the Argon2id string, never the password) and `created_at`. The `trades` table stores `id`, `user_id`, `ticker`, `direction`, `entry_price`, `exit_price`, `quantity`, `fees`, `trade_date` and `notes`. Prices are REAL, quantity is INTEGER and dates are ISO-8601 TEXT (YYYY-MM-DD), which sort alphabetically in chronological order. The index `idx_trades_user_date` on `(user_id, trade_date)` supports the dominant access pattern of fetching one user's trades in date order.

Table 3.4: Constraint summary

Column	Constraint	Purpose
users.id, trades.id	PRIMARY KEY AUTOINCREMENT	Unique row identity
users.username	UNIQUE NOT NULL	No duplicate accounts
users.password_hash	NOT NULL	A stored hash is always present
trades.user_id	NOT NULL, FOREIGN KEY ON DELETE CASCADE	Every trade belongs to a user; deleting a user removes their trades
trades.direction	CHECK in ('Long', 'Short')	Only valid directions
entry_price, exit_price	CHECK > 0	Positive prices
trades.quantity	CHECK > 0	Positive integer quantity
trades.fees	CHECK >= 0	Fees cannot be negative
trades.trade_date	NOT NULL	Date present; format enforced by validators

SQLite checks foreign keys only after `PRAGMA foreign_keys = ON` is issued, and the setting applies to each connection separately; the application enables it for its connection. All user-supplied values are bound to prepared statements with `sqlite3_bind_*` and never

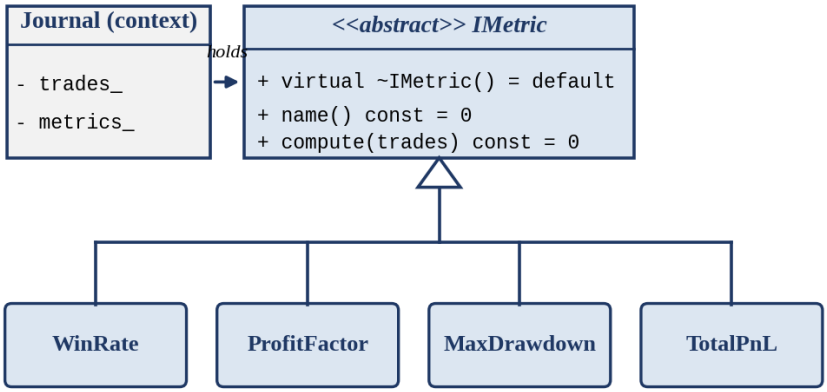
concatenated into SQL text, and every trade query contains `WHERE user_id = ?`, so ownership is enforced inside the repository rather than left to the route handlers.

3.6 Object-Oriented Class Design

Table 3.5: Principal classes and their responsibilities

Class / interface	Kind	Responsibility
`Trade`	Value object	One executed trade; `profit()`; `enum class Direction`; `operator<`, `==`, `<<`.
`TradeFilter`	Value struct	Carries ticker and outcome filter values.
`IMetric`	Abstract interface	Pure virtual `name()` and `compute()`; the Strategy interface.
`WinRate`, `ProfitFactor`, `MaxDrawdown`, `TotalPnL`	Concrete strategies	Implement `IMetric`; one statistic each.
`Journal`	Context (move-only)	Owns trades and metric strategies; `computeAll()`, equity curve.
`AppError` and derived	Exception hierarchy	`ValidationError` and `DatabaseError` derive from `AppError`, which extends `std::runtime_error`.
`Database`, `Statement`	RAII wrappers	Own `sqlite3*` and `sqlite3_stmt*`; release them in destructors.
`ITradeRepository`	Abstract interface	Storage contract for trades.
`SqliteTradeRepository`, `InMemoryTradeRepository`	Implementations	Production SQLite storage and in-memory mock.
`UserRepository`	Data access	User lookup and registration.
`AuthService`	Service	Argon2id hashing and verification through libsodium.
`AuthGuard`	Middleware	Session check, redirection of unauthenticated visitors.
`Deps`	Dependency holder	Bundle of repositories and services passed to routes.

Strategy pattern. The Strategy pattern defines a family of algorithms behind one interface so that they can be exchanged without changing the code that uses them. Here `IMetric` is the interface, the metric classes are the strategies and `Journal` is the context (Figure 3.4). `Journal::computeAll()` loops over its metrics and records the pair (name, value) for each, without knowing which concrete metric it is calling. The dashboard receives a list of named values, so adding a new metric means adding one new class and nothing else.



Journal::computeAll() calls name() and compute() on every strategy through the IMetric pointer; it never needs the concrete type.

Figure 3.4: Strategy pattern structure

Repository pattern. The Repository pattern places a collection-like interface between business logic and storage so that the domain deals with `Trade` objects, not SQL. Route handlers receive `ITradeRepository` through `Deps`, so the concrete class is chosen in one place, `main.cpp`. This is dependency inversion: high-level modules depend on an abstraction, and the low-level SQLite code also depends on that abstraction by implementing it (Figure 3.5).

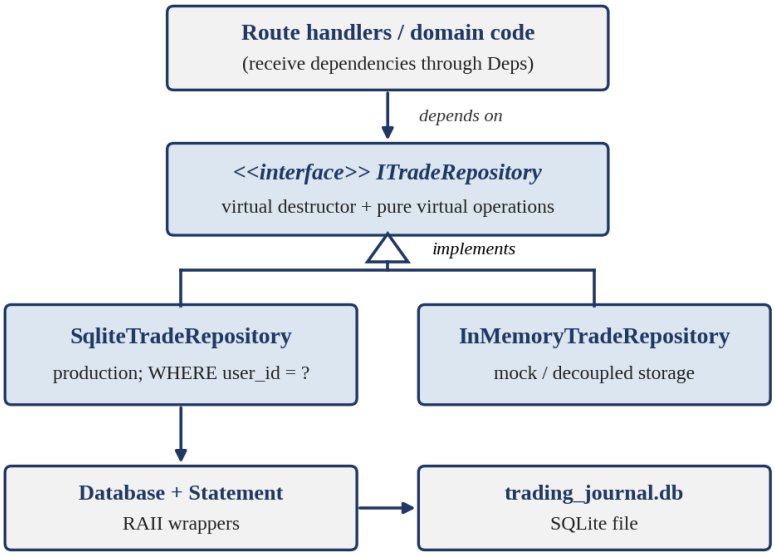


Figure 3.5: Repository pattern structure

3.7 Mathematical Design of the Analytics

The analytics engine processes trades in date order; `Trade::operator<` orders trades chronologically so that `std::sort` can be used. Let P_i be the profit of the i-th trade.

Trade profit. Long: Profit = (Exit − Entry) × Quantity − Fees. Short: Profit = (Entry − Exit) × Quantity − Fees. Fees are subtracted in both cases, so a trade with a small price gain can still be a net loss. Example (Long): Entry 100, Exit 110, Quantity 50, Fees 20 gives (110 − 100) × 50 − 20 = 480. Example (Short): Entry 200, Exit 190, Quantity 10, Fees 5 gives (200 − 190) × 10 − 5 = 95.

Win Rate = (N_wins / N_total) × 100, the percentage of trades that finished in profit.

Profit Factor = Gross Profits / |Gross Losses|. A value above 1 means profits exceeded losses over the period. If gross losses are zero the ratio is undefined; the implementation returns the sentinel −1.0, which is intended to be displayed as "N/A".

Equity curve. $E_k = \Sigma P_i$ for i = 1 to k, with $E_0 = 0$; it is the running total of profit after each trade.

Maximum drawdown. $Peak_k$ = max of E_i up to step k; $DD_k = Peak_k - E_k$; MDD = max of DD_k. The implementation computes it in a single pass with running equity, peak and the largest drop, starting the peak at 0.0 (the starting capital counted as the first peak).

Average Win / Average Loss. The mean profit of winning trades and the mean loss (as a magnitude) of losing trades.

3.8 Security Design

Table 3.6: Security measures in the design (local HTTP use)

Measure	How it works	Where
Argon2id password hashing	`crypto_pwhash_str` gives a salted, memory-hard hash; only `password_hash` is stored	auth_service.cpp
SQL injection prevention	Values bound by `sqlite3_bind_*`; no string interpolation of user input	Statement, repositories
Session authentication	`AuthGuard` checks the session and redirects unauthenticated users	middleware.cpp
HTTP-only cookies	The session cookie carries the HttpOnly attribute, so page scripts cannot read it	middleware.cpp
User-specific queries	`WHERE user_id = ?` on reads, updates and deletions	trade_repository.cpp
Output escaping	Mustache `{{ var }}` escapes by default, mitigating XSS	templates/
Input validation	Server-side validators plus CHECK constraints	validators.cpp, schema.sql
Resource cleanup	RAII for `sqlite3*` and `sqlite3_stmt*`	Database, Statement

4. IMPLEMENTATION

4.1 Project Directory Structure

The project root is `trading_journal/`. Headers (the contracts) live in `include/`, implementations in `src/`, views in `templates/`, browser assets in `static/`, SQL scripts in `sql/`, and the bundled Crow header in `third_party/crow/`. The letters A, B and C mark the team member who owns each file.

```
trading_journal/
+-- CMakeLists.txt, demo.sh, README.md, .gitignore       [C]
+-- include/
|   +-- errors.hpp, trade.hpp, metrics.hpp, journal.hpp, validators.hpp          [A]
|   +-- database.hpp, statement.hpp, user_repository.hpp,
|   |   trade_repository.hpp, in_memory_trade_repository.hpp                     [B]
|   +-- auth_service.hpp, middleware.hpp, routes.hpp                             [C]
+-- src/
|   +-- trade.cpp, metrics.cpp, journal.cpp, validators.cpp,
|   |   routes_analytics.cpp                                                     [A]
|   +-- database.cpp, statement.cpp, user_repository.cpp,
|   |   trade_repository.cpp, in_memory_trade_repository.cpp                     [B]
|   +-- auth_service.cpp, middleware.cpp, routes_auth.cpp,
|       routes_trades.cpp, main.cpp                                              [C]
+-- templates/  base, login, register, trades, trade_form .html [C]; dashboard.html
[A]
+-- static/     css/style.css [C];  js/charts.js [A]
+-- sql/        schema.sql [B];  seed.sql [A]
+-- third_party/crow/crow_all.h                                    [external]
```

4.2 Module Organisation and File Responsibilities

Table 4.1: File responsibilities

File(s)	Responsibility	Owner
errors.hpp	Exception hierarchy: `AppError`, `ValidationError`, `DatabaseError`	A
trade.hpp / .cpp	`Trade` entity, `Direction` enum, `TradeFilter`, profit calculation, operators, string conversion	A
metrics.hpp / .cpp	`IMetric` and the four concrete metrics	A
journal.hpp / .cpp	Move-only `Journal`, `computeAll()`, equity curve, `sumProfit` template	A
validators.hpp / .cpp	Server-side input validation	A
routes_analytics.cpp, dashboard.html, charts.js	Dashboard route, equity-curve JSON API, KPI cards and Chart.js rendering	A
seed.sql	20 historical trades for demonstration	A
database.*, statement.*	RAII wrappers for `sqlite3*` and `sqlite3_stmt*`; binding, stepping, column extraction	B
trade_repository.*, in_memory_trade_repository.*	`ITradeRepository`, SQLite implementation with user isolation, in-memory implementation	B
user_repository.*, schema.sql	User lookup and registration; tables, constraints and index	B
auth_service.*, middleware.*	Argon2id hashing and verification; `AuthGuard` and session cookies	C
routes.hpp, routes_auth.cpp, routes_trades.cpp	`Deps`, registration, login, logout, trade CRUD, filtering, CSV export	C

File(s)	Responsibility	Owner
main.cpp, CMakeLists.txt, demo.sh	Entry point and composition of `Deps`; build configuration; demonstration script	C
base, login, register, trades, trade_form .html; style.css	Layout and views	C

4.3 Domain Layer Implementation

Exceptions. `errors.hpp` defines `AppError`, which extends `std::runtime_error`, and the two derived types `ValidationError` and `DatabaseError`. Errors are signalled by exceptions instead of error codes, so the code that detects a problem is separate from the code that handles it.

The Trade class. `Trade` models one executed trade as a value object. Its direction is the strongly typed `enum class Direction { Long, Short }`, with static helpers to convert between the enumeration and the text stored in the database. The const method `Trade::profit()` applies the Long or Short formula of Section 3.7 including fees. `operator<` orders trades by date, `operator==` compares them and `operator<<` prints them for logging.

Validators. `validators.cpp` checks every form field on the server and throws `ValidationError` on failure. The database repeats the key rules as CHECK constraints (price > 0, quantity > 0, fees >= 0), which gives two independent lines of defence.

Metrics (Strategy pattern). The interface and one concrete strategy are shown below. `MaxDrawdown` is marked `final`, so no further class can derive from it, and both functions are `const` so a calculation can never modify the data.

```
class IMetric {
public:
    virtual ~IMetric() = default;
    virtual std::string name() const = 0;
    virtual double compute(const std::vector<Trade>& trades) const = 0;
};

class MaxDrawdown final : public IMetric {
public:
    std::string name() const override { return "Max Drawdown"; }
    double compute(const std::vector<Trade>& trades) const override {
        double equity = 0.0, peak = 0.0, maxDd = 0.0;
        for (const auto& trade : trades) {
            equity += trade.profit();
            peak   = std::max(peak, equity);
            maxDd  = std::max(maxDd, peak - equity);
        }
        return maxDd;
    }
};
```

4.4 Journal and Template-Based Aggregation

`Journal` is the context of the Strategy pattern. It holds a `std::vector<Trade>` and a `std::vector<std::unique_ptr<IMetric>>`. Because a `unique_ptr` cannot be copied,

`Journal` is a **move-only** type: its copy constructor and copy assignment are deleted, its move operations are `noexcept = default`, and its destructor is defaulted, which is the Rule of Five written out explicitly. `computeAll()` calls `metric->name()` and `metric->compute(trades_)` on every strategy, so identical code runs different formulas (runtime polymorphism).

The function template `sumProfit` in `journal.hpp` is a generic accumulator. It is instantiated with lambda predicates to select gross profits and gross losses, from which the Profit Factor is derived. The equity curve is the running sum of `profit()` over the date-ordered trades; `routes_analytics.cpp` exposes it as JSON and `charts.js` draws it. The full `Journal` declaration is given in Appendix C.

4.5 Data Layer and RAII Wrappers

The SQLite C API requires every connection and statement to be closed or finalised exactly once. Two small classes make that automatic. `Database` owns the `sqlite3*` connection and closes it in its destructor. `Statement` owns the `sqlite3_stmt*`, throws `DatabaseError(sqlite3_errmsg(db))` if preparation fails, finalises the statement in its destructor and offers `bindInt`, `bindText` and `step`. Copying is deleted, because a copied raw pointer would be finalised twice, while the move constructor transfers the pointer and sets the source to `nullptr`.

```
class Statement {
public:
    Statement(sqlite3* db, const std::string& sql) {
        if (sqlite3_prepare_v2(db, sql.c_str(), -1, &stmt_, nullptr) != SQLITE_OK)
            throw DatabaseError(sqlite3_errmsg(db));
    }
    ~Statement() { if (stmt_) sqlite3_finalize(stmt_); }
    Statement(const Statement&) = delete;
    Statement& operator=(const Statement&) = delete;
    Statement(Statement&& other) noexcept : stmt_(other.stmt_) { other.stmt_ = nullptr; }
    // bindInt(), bindText(), step() ...
private:
    sqlite3_stmt* stmt_{nullptr};
};
```

`SqliteTradeRepository` and `InMemoryTradeRepository` both implement `ITradeRepository`, which has a virtual destructor and pure virtual operations. Lookups return `std::optional<Trade>`, so an absent trade is expressed in the type instead of a null pointer or a magic value. `UserRepository` handles user retrieval and registration.

4.6 Web and Security Layer

`main.cpp` is the composition root. It creates the `Database`, the repositories and the `AuthService` once, places them in the `Deps` structure declared in `routes.hpp`, binds the routes and starts the Crow server on port 18080. Passing dependencies this way is **dependency injection**: handlers receive interfaces and never construct their own storage.

`AuthService` wraps libsodium's `crypto_pwhash_str` and its verification counterpart, converting C-style return codes into exceptions. `AuthGuard` is Crow middleware that checks the session cookie (HttpOnly) on protected routes and redirects unauthenticated visitors. The route groups are `/`, `/login`, `/register`, `/trades` and `/export`. `routes_trades.cpp` implements trade entry, editing, the ledger, filtering and the RFC 4180 CSV export, which quotes fields that contain commas or quotes so that the file opens correctly in Excel, Google Sheets or R. Views are Mustache templates, so `{{ var }}` output is HTML-escaped by default.

4.7 Object-Oriented Concepts Applied

Table 4.2: OOP concept-to-code mapping

No.	Concept	Where it appears in the project
1	Classes and objects	`Trade`, `Journal`, `Database`, `Statement`
2	Encapsulation	Private state in `Trade`, `Journal`, `Database`; const getters
3	Abstraction	`IMetric`, `ITradeRepository`
4	Inheritance	`MaxDrawdown : IMetric`; `ValidationError` and `DatabaseError` from `AppError` from `std::runtime_error`
5	Runtime polymorphism	`Journal::computeAll()` calls `IMetric` through a base pointer
6	Pure virtual functions	`IMetric::name`, `IMetric::compute`; `ITradeRepository`
7	Const-correctness	`Trade::profit()`, `IMetric::compute()`
8	enum class	`Trade::Direction`
9	Static helper functions	Direction-to-text conversion in `Trade`
10	Virtual destructors	`virtual ~IMetric() = default;`, `ITradeRepository`
11	Smart pointers	`std::unique_ptr<IMetric>` in `Journal`; `std::shared_ptr<Database>`
12	Rule of Five, move semantics, deleted copy	`Journal`, `Statement`, `Database`
13	Operator overloading	`Trade::operator<`, `operator==`, `operator<<`
14	Function templates and lambdas	`sumProfit()` with lambda predicates in `journal.hpp`
15	Exception handling	`errors.hpp`; `Statement` constructor throws `DatabaseError`
16	RAII	`Database`, `Statement`
17	std::optional	Repository lookups, for example `std::optional<Trade>`
18	C-library interoperability	`Statement` (SQLite C API), `AuthService` (libsodium)
19	Dependency injection	`Deps` created in `main.cpp`, declared in `routes.hpp`
20	Strategy and Repository patterns	`IMetric` family with `Journal`; `ITradeRepository` family

4.8 Build and Execution

Prerequisites (Ubuntu, Debian or WSL):

```
sudo apt update
sudo apt install build-essential cmake libsqlite3-dev libsodium-dev libasio-dev
sqlite3
```

macOS (Homebrew): `brew install cmake sqlite libsodium asio`. **Automated run:** `./demo.sh` compiles the project, creates the schema, loads the 20 seed trades and starts the server. **Manual run:**

```
cmake -B build -S . -DCMAKE_BUILD_TYPE=Release
cmake --build build -j4
sqlite3 trading_journal.db < sql/schema.sql
sqlite3 trading_journal.db < sql/seed.sql
./build/trading_journal
```

Optional sanitizer build: `cmake -B build_san -S . -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined -g"`, then `cmake --build build_san -j4` and `./build_san/trading_journal`. The application is opened at `http://localhost:18080`, which is a local address and does not mean the application is hosted online.

4.9 Team Contributions

The work was divided into three modules that meet at defined interfaces.

Table 4.3: Responsibility matrix

Member	Roll number	Module and main work
Utkarsh Kumar (Member A)	25008288 (41)	Domain logic and analytics: exception hierarchy, `Trade`, the `IMetric` strategies, move-only `Journal` with `computeAll()`, equity curve and `sumProfit`, validators, analytics routes, `dashboard.html`, `charts.js` and `seed.sql`.
Pinkesh Kumar Mishra (Member B)	25008253 (07)	Data persistence: RAII `Database` and `Statement`, `ITradeRepository` with the SQLite and in-memory implementations, `UserRepository`, and `schema.sql` with constraints, foreign key and index.
Rabeekant (Member C)	25008259 (13)	Web infrastructure and security: Crow server and `Deps` in `main.cpp`, registration, login and logout, Argon2id `AuthService`, `AuthGuard`, trade routes with filtering and CSV export, templates and CSS, `CMakeLists.txt` and `demo.sh`.

Integration works as follows: Member B's repositories return and accept Member A's `Trade` objects; Member A's analytics routes load a user's trades through the repository and pass them to a `Journal`; Member C's `main.cpp` creates Member B's objects and Member C's `AuthService`, places them in `Deps` and passes them to every route handler, while `AuthGuard` protects those routes.

5. RESULT

5.1 Features Delivered

The implemented application provides the following functionality, as documented in the project:

Table 5.1: Features and the modules that provide them

Feature	What the user obtains	Main modules
Registration, login, logout	Account with Argon2id-hashed password; session by HttpOnly cookie	routes_auth, auth_service, middleware
Trade entry and editing	One form (`trade_form.html`) for adding and editing	routes_trades, validators
Trade ledger and filtering	Table with outcome badges; filter by ticker and by wins or losses	routes_trades, trades.html
Long and Short profit	Correct profit including fees for both directions	trade.cpp
Dashboard KPIs	Total P&L, Win Rate, Profit Factor, Max Drawdown, Average Win, Average Loss	journal, metrics, dashboard.html
Equity curve	Line chart of cumulative profit drawn by Chart.js	journal, routes_analytics, charts.js
CSV export	RFC 4180 file containing only the user's own trades	routes_trades
Data isolation	Every query filtered by `user_id`	trade_repository
Persistence	Data survives restarts in `trading_journal.db`	database, repositories, schema

“Real-time” on the dashboard means that the figures are calculated from the stored trades each time the page is requested; no live market data is used.

5.2 Worked Example of the Analytics

To show the results that the analytics engine is designed to produce, consider six trades in date order with profits +200, −80, +120, −150, −50 and +300. The values below were calculated

from the formulas of Section 3.7; they are a numerical illustration of the algorithms and not output captured from a run of the application.

Table 5.2: Illustrative trade series, equity and drawdown

k	Profit P_k	Equity E_k	Peak$_k$	Drawdown DD_k
0	–	0	–	–
1	+200	200	200	0
2	−80	120	200	80
3	+120	240	240	0
4	−150	90	240	150
5	−50	40	240	200
6	+300	340	340	0

Table 5.3: Dashboard metrics for the illustrative series

Metric	Calculation	Result
Total P&L	200 − 80 + 120 − 150 − 50 + 300	340
Win Rate	(3 winners / 6 trades) × 100	50 %
Profit Factor	Gross profits 620 / gross losses 280	2.21
Maximum Drawdown	Peak 240 falls to 40 at k = 5	200
Average Win	620 / 3	206.67
Average Loss	280 / 3 (magnitude)	93.33

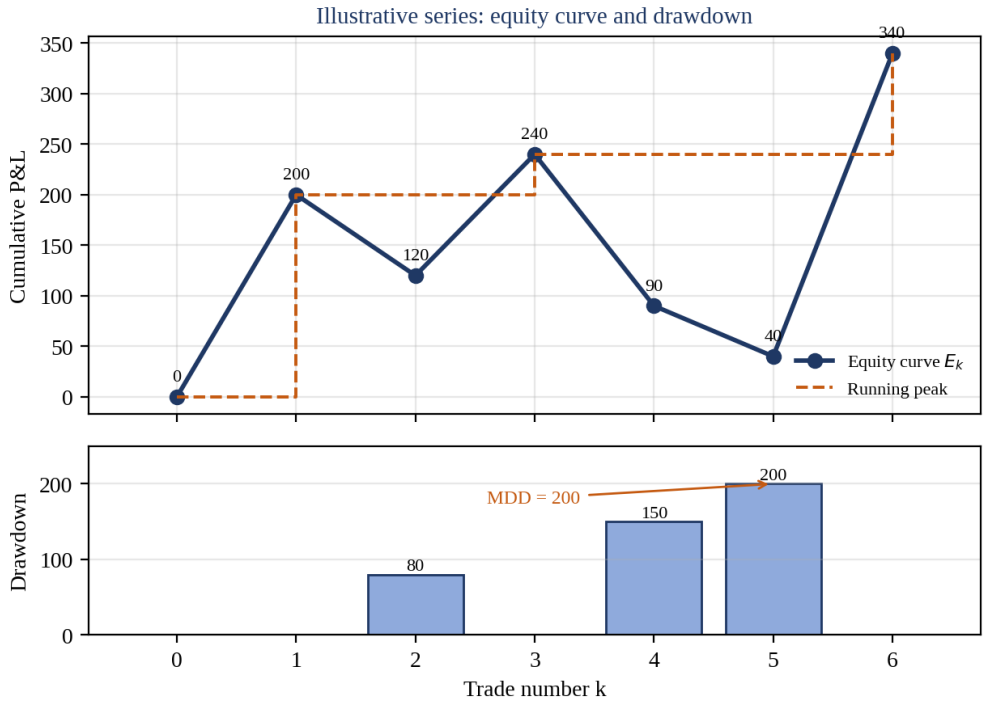


Figure 5.1: Equity curve and drawdown for the illustrative series

The equity curve rises to 240, falls to 40 and finishes at 340. The net profit of 340 equals gross profits minus gross losses (620 − 280), which cross-checks the Profit Factor calculation. The same trade history shows a 50 % Win Rate but a Profit Factor above 2, which illustrates why a single statistic is not enough: the winners are larger than the losers even though only half the trades win.

5.3 Test Plan and Expected Results

The following test cases cover the main features. The "Expected result" column follows from the design and from the worked example above; the "Observed result" column is to be completed by the team after running each case, replacing the blank with Pass or Fail and the actual behaviour.

Table 5.4: Test plan

ID	Feature	Input or action	Expected result	Observed
TC-01	Registration	New valid username and password	Account created; only a hash stored	✅
TC-02	Duplicate user	Register the same username twice	Second attempt rejected	✅
TC-03	Login failure	Correct user, wrong password	Access denied	✅
TC-04	Logout and guard	Log out, open a protected page	Redirected to login	✅
TC-05	Long profit	Entry 100, Exit 110, Qty 50, Fees 20	Profit = 480	✅
TC-06	Short profit	Entry 200, Exit 190, Qty 10, Fees 5	Profit = 95	✅
TC-07	Invalid input	Price 0, quantity 0 or negative fees	Rejected by validator and by CHECK	✅
TC-08	Filtering	Filter by one ticker; by Wins; by Losses	Only matching trades shown	✅
TC-09	Dashboard metrics	Enter the series of Table 5.2	340; 50 %; about 2.21; 200	✅
TC-10	Profit Factor, no losses	Only winning trades	Internal −1.0 shown as N/A	✅
TC-11	CSV export	Note containing a comma and a quote	Valid RFC 4180 file, own trades only	✅
TC-12	Data isolation	User B opens user A's trade	Denied or not found	✅
TC-13	Cascade delete	Delete a user row with foreign keys ON	That user's trades removed	✅
TC-14	Persistence	Stop and restart the server	Data still present	✅
TC-15	Output escaping	Save a note containing a script tag	Shown as text, not executed	✅
TC-16	Sanitizers	Run the sanitizer build through all features	No ASan or UBSan report	✅

No test outcome is claimed in this report beyond the arithmetic of Section 5.2. Test evidence (real screenshots, the sanitizer log with compiler version and date, and `sqlite3` query output for TC-13) should be attached by the team.

5.4 Discussion

The result of the project is a working architecture in which every OOP concept listed in Table 4.2 has a concrete place in the code, and in which each layer can be understood on its own. The domain layer calculates every figure without knowing about HTTP or SQL; the data layer hides SQLite behind an interface; and the web layer only translates between the browser and these layers. The worked example shows that the analytics are simple to verify by hand, which makes the test plan straightforward to execute.

6. CONCLUSION

The Trading Journal is a multi-user web application in C++17 that lets a trader record trades, review them in a ledger, filter them, view Total P&L, Win Rate, Profit Factor, Maximum Drawdown, Average Win and Average Loss, see an equity curve and export data to CSV. Its practical value is that it replaces ad hoc notes with validated, consistently calculated records that the trader can use to review his or her own past results. It does not guarantee profits and does not provide recommendations.

The project combines financial analytics and web development. Calculations are performed in a domain layer that knows nothing about HTTP or SQL, while Crow, Mustache, Bootstrap and Chart.js present the results. Object-oriented concepts are applied to real needs: an `IMetric` interface with polymorphic strategies, an `ITradeRepository` abstraction with two implementations, RAII wrappers for SQLite handles, move-only types, templates and lambdas, custom exceptions and dependency injection. The security design adds Argon2id hashing, parameterized queries, a session guard and user-scoped queries.

For the team, the project showed how three separately written modules can be integrated through agreed interfaces: the domain and analytics module, the persistence module, and the web and security module. Remaining work is to record test evidence for the memory-safety, security and correctness goals, to settle the maximum-drawdown convention (Appendix D), and to extend the system as described in Chapter 7.

7. SCOPE AND LIMITATIONS

7.1 Scope of the Project

Within scope: registration, login and logout; a trade ledger with add, edit and view operations; Long and Short directions; filtering by ticker and by outcome; dashboard metrics; the equity curve; CSV export; user-specific data isolation; and SQLite persistence.

Outside scope: live or delayed market prices, broker connectivity, automated trading, predictions, investment recommendations, multi-currency conversion, and complex derivatives such as option Greeks or multi-leg spreads.

7.2 Limitations

- **Single-currency assumption.** All entries are assumed to be in one base currency (for example USD or INR); there is no exchange-rate conversion.
- **Asset scope.** The system is aimed at cash equities and linear instruments; non-linear derivatives are not modelled.
- **Local HTTP execution.** The application is designed for local use on `localhost:18080` over HTTP. Production use would require TLS/HTTPS termination and CSRF protection, so it should not be described as production-ready.
- **Recorded history only.** The dashboard analyses only the trades a user has entered; it uses no market data..
- **Single-database design.** SQLite suits a local application; high-concurrency production use would need a different engine, as the interface-based design makes possible.

7.3 Future Enhancements

Table 7.1: Possible future enhancements (not implemented)

Enhancement	How it could extend the system
Sharpe and Sortino ratios	New `IMetric` implementations; they need a defined return series and assumptions that the current trade-level data does not state.
Profit-to-MaxDD ratio	A metric relating profit to the maximum drawdown already computed.
Win/loss streaks	A metric scanning the date-ordered trades for runs of wins or losses.
Emotional and strategy tagging	New schema fields, form inputs and filters for states such as Disciplined, FOMO or Revenge.
WebSocket market feeds	Real-time prices for valuing open positions; adds a network component and an external data dependency.
PostgreSQL migration	A new repository implementation behind `ITradeRepository`; SQL differences handled in that layer.
Docker and Docker Compose	A container definition to build and run the server without manual installation of dependencies.

REFERENCES

1. ISO/IEC 14882:2017, Programming Languages: C++ (the C++17 standard), International Organization for Standardization.
2. Crow C++ microframework. https://crowcpp.org and https://github.com/CrowCpp/Crow
3. SQLite documentation. https://www.sqlite.org/docs.html
4. libsodium documentation, password hashing. https://doc.libsodium.org
5. CMake documentation. https://cmake.org/documentation/
6. Bootstrap 5 documentation. https://getbootstrap.com/docs/
7. Chart.js documentation. https://www.chartjs.org/docs/
8. Mustache template specification. https://mustache.github.io/
9. RFC 4180, Common Format and MIME Type for CSV Files. https://www.rfc-editor.org/rfc/rfc4180
10. AddressSanitizer. https://clang.llvm.org/docs/AddressSanitizer.html and UndefinedBehaviorSanitizer. https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html
11. Project README.md, Trading Journal: High-Performance C++17 Web Application (team document).
12. Project source code repository (GitHub): https://github.com/Utkarsh-debug12/Trading-Journal.git

Web references should be opened and checked by the team, and the access date added, before submission.

APPENDIX

Appendix A: SQL Schema (sql/schema.sql)

```
CREATE TABLE IF NOT EXISTS users (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    username      TEXT UNIQUE NOT NULL,
    password hash TEXT NOT NULL,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE IF NOT EXISTS trades (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER NOT NULL,
    ticker      TEXT NOT NULL,
    direction   TEXT NOT NULL CHECK(direction IN ('Long', 'Short')),
    entry_price REAL NOT NULL CHECK(entry_price > 0),
    exit_price  REAL NOT NULL CHECK(exit_price > 0),
    quantity    INTEGER NOT NULL CHECK(quantity > 0),
    fees        REAL DEFAULT 0.0 CHECK(fees >= 0),
    trade_date  TEXT NOT NULL,   -- ISO-8601 (YYYY-MM-DD)
    notes       TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
CREATE INDEX IF NOT EXISTS idx_trades_user_date ON trades(user_id, trade_date);
```

Appendix B: Important Formulas

- Profit (Long) = (Exit − Entry) × Quantity − Fees
- Profit (Short) = (Entry − Exit) × Quantity − Fees
- Win Rate = (N_wins / N_total) × 100
- Profit Factor = Σ Gross Profits / Σ |Gross Losses| (returns −1.0, shown as N/A, if gross loss is zero)
- Equity: $E_k = \Sigma P_i$ (i = 1 to k), $E_0 = 0$
- $Peak_k$ = max of E_i; $DD_k = Peak_k - E_k$; MDD = max of DD_k

Appendix C: Selected C++ Excerpts

C.1 Journal (include/journal.hpp)

```
class Journal {
public:
    explicit Journal(std::vector<Trade> trades);
    Journal(const Journal&) = delete;
    Journal& operator=(const Journal&) = delete;
    Journal(Journal&&) noexcept = default;
    Journal& operator=(Journal&&) noexcept = default;
    ~Journal() = default;
    std::vector<std::pair<std::string, double>> computeAll() const {
        std::vector<std::pair<std::string, double>> results;
        for (const auto& metric : metrics_)
            results.emplace_back(metric->name(), metric->compute(trades_));
        return results;
    }
private:
    std::vector<Trade> trades_;
    std::vector<std::unique_ptr<IMetric>> metrics_;
};
```

C.2 Statement (include/statement.hpp)

```
class Statement {
public:
    Statement(sqlite3* db, const std::string& sql) {
```

```
        if (sqlite3_prepare_v2(db, sql.c_str(), -1, &stmt_, nullptr) != SQLITE_OK) {
            throw DatabaseError(sqlite3_errmsg(db));
        }
    }
    ~Statement() { if (stmt_) sqlite3_finalize(stmt_); }
    Statement(const Statement&) = delete;
    Statement& operator=(const Statement&) = delete;
    Statement(Statement&& other) noexcept : stmt_(other.stmt_) { other.stmt_ = nullptr; }
    void bindInt(int index, int value) { sqlite3_bind_int(stmt_, index, value); }
    void bindText(int index, const std::string& val) {
        sqlite3_bind_text(stmt_, index, val.c_str(), -1, SQLITE_TRANSIENT);
    }
    bool step() { return sqlite3_step(stmt_) == SQLITE_ROW; }
private:
    sqlite3_stmt* stmt_{nullptr};
};
```

Appendix D: Points to Confirm Before Final Submission

The following details are not fully specified in the project documentation and should be checked against the source code:

- **Drawdown convention.** The formula takes the peak over E_1 to E_k, while the code starts the peak at 0.0. They differ when the history begins with a loss (for P = −50, +20 the formula gives MDD = 0 and the code gives 50). State one convention explicitly.
- **Profit Factor with zero losses:** confirm that −1.0 is displayed as “N/A” and never as −1.0.
- **Delete operation:** confirm that a delete route and button exist in the interface.
- **Zero-profit trades** and the case of no trades: how they are classified and what Win Rate returns.
- **Average Win / Loss:** where they are computed and the sign convention.
- **Route paths and HTTP methods,** including the equity-curve JSON endpoint.
- **`PRAGMA foreign_keys = ON`** is issued for every connection opened in `database.cpp`.
- **Bootstrap and Chart.js** are served locally or from a CDN (affects offline use).
- **Sanitizer evidence:** attach the real log, compiler version, commands and date.
- **Minimum compiler version** and hardware specification.

GITHUB REPOSITORY

The complete source code, SQL scripts, templates, build files and the project README of the Trading Journal are available in the project repository on GitHub:

Repository link: https://github.com/Utkarsh-debug12/Trading-Journal.git

To obtain and run the project:

```
git clone https://github.com/your-username/trading_journal.git
cd trading_journal
./demo.sh
```

After the server starts, open `http://localhost:18080` in a web browser.